SINGAPORE UNIVERSITY OF SOCIAL SCIENCES

ICT133

Proctored Online Exam - January Semester 2026

Structured Programming

Thursday, 16 April 2026 **01:00 pm - 03:00 pm**

Time allowed: 2 hours

INSTRUCTIONS TO STUDENTS:

1. This examination contains **10** questions.
2. Answer all parts of each question in the corresponding answer box.
3. All answers must be typed in the answer box and submitted in Examena.
4. This is a/an **Closed-book** examination.
5. To prevent plagiarism and collusion, your submission will be reviewed by Turnitin. The Turnitin report will only be made available to the marker and you will not be able to view it.
6. The University takes plagiarism and collusion seriously, and your Turnitin report will be examined thoroughly as part of the marking process.
7. **One (1)** A4-sized (double-sided) hardcopy Cheat Sheet allowed.

At the end of the examination.
To end the examination, please click on the "Submit and end exam" button to submit your answers.

THE UNIVERSITY RESERVES THE RIGHT NOT TO MARK YOUR SCRIPT IF YOU FAIL TO FOLLOW THESE INSTRUCTIONS.

(Full marks: 100)

Question 1

Apply the principles of structured programming principles in answering questions 1-5.

The character that must be at the end of the line for if, while, for statements in Python is:

A. *
B. #
C. :
D. \

(5 marks)

Question 2

What is the output when the function testSelection() is called?

```
def testSelection():
    a = 3
    b = 5
    c = 4

    if a > b:
        print("one")

    if a < c:
        print("two")
    elif c < b:
        print("three")
```

A. one
B. two
C. three
D. one three

(5 marks)

Question 3

What will be the output when the given code is executed?

```
i = 0
while True:
    if i % 3 == 0:
        break

    i = i * 3
    print(i)
```

A. 0
B. Infinite loop
C. No output
D. 3

(5 marks)

Question 4

What will be the output of the following Python code?

```
x = [1, 2, 3]
y = x
y[0] = 99
print(x)
```

A. [1, 2, 3]
B. [99, 2, 3]
C. [1, 99 , 3]
D. Error

(5 marks)

Question 5

What will be the output of the following Python code?

```
my_dict = {"a": 1, "b": 2}
my_dict["c"] = my_dict.get("a", 0) + my_dict.get("d", 3)
print(my_dict)
```

A. { "a": 1, "b": 2, "c": 4 }
B. { "a": 1, "b": 2, "c": 1 }
C. { "a": 1, "b": 2, "c": 3 }
D. Error

(5 marks)

Question 6

The EV charging rates at a city charging hub are as follows:

Charger Type	Tier	Rates
DC Fast		$0.68 per kWh
AC Standard	Tier 1: Less than 20 kWh	$0.56 per kWh
	Tier 2: Next 20 – 50 kWh	$0.54 per kWh
	Tier 3: Any energy above 50 kWh	$0.52 per kWh

Assume energy is measured as kWh delivered to the vehicle. Users may charge any decimal amount (e.g., 18.75 kWh).

Develop a Python program that:

- Prompts for a charger type (AC or DC).
- Validates the charger type: if input is not AC or DC, keep prompting until a valid charger type is entered.
- For a valid charger type, prompt for energy (kWh) as a non-negative number.
- Calculates and displays the amount due:
  - DC Fast: amount = 0.68 × kWh
  - AC Standard (tiered): apply tiered rates to portions of the energy as defined above.
- Assume user input the correct data type for all input.

Sample Run 1:

Enter charger type (DC/AC): **AC**

Energy delivered (kWh): **28**

Please pay $15.12

Sample Run 2:

Enter charger type (DC/AC): **DC**

Energy delivered (kWh): **15.5**

Please pay $10.54

Sample Run 3:

Enter charger type (DC/AC): **CDC**

Wrong charger type, please enter AC or DC

(15 marks)

Question 7

You are given a text file named input.txt that contains multiple paragraphs of English text. Each paragraph spans several lines. Each line may contain lowercase and uppercase letters, punctuation, digits and spaces.

Develop a Python program that reads the file and determines the total number of characters, ignoring spaces.

Suppose input.txt contains:

The quick brown fox jumps over the 2 lazy dogs.
One dog barked back at the fox; the other dog ran away.

Your program should output:

Total character count: 82

(15 marks)

Question 8

Develop a function *mergeDictionary(dict1, dict2) -> dict* to solve the following problem: merging two dictionaries by adding the values of matching keys. If a key exists in only one dictionary, include it as-is in the result.

Constraints:

- Do not use set, collections.Counter, or any external libraries.
- You may use only lists, dictionaries, and basic control structures (for, if, etc.).

Input:

- Two dictionaries dict1 and dict2 with string keys and integer values.

The function returns a new dictionary with all keys from both inputs. If a key appears in both, its value should be the sum of the values from dict1 and dict2.

Example:

```
dict1 = {'a': 10, 'b': 20, 'c': 30}
dict2 = {'b': 5, 'c': 15, 'd': 25}
print( mergeDictionary(dict1, dict2) )
```

Output:

```
{'a': 10, 'b': 25, 'c': 45, 'd': 25}
```

Algorithm Guide:
- Initialize an empty dictionary called result.
- Iterate through all key-value pairs in dict1:
- Add each key and value to result.
- Iterate through all key-value pairs in dict2:
- If the key already exists in result, add the value to the existing one.

- If the key does not exist, insert it into result with its value.
- Return the result dictionary.

(15 marks)

Question 9

Develop a Python function named *openMysteryBox() -> int* that simulates a game where a player opens mystery boxes to accumulate gold coins. Each box contains either:

- A random number of coins between 1 and 6, or
- A bomb (represented by -1), which resets the player's total coins to 0.

The player starts with 0 coins, and the goal is to reach exactly 25 coins.

Function Requirements:

- The function must be named *openMysteryBox()*
- When a player opens a mystery box, use the random.randint(-1, 6) to simulate the number of gold coins in each box:
    - -1 represents drawing a bomb, the total resets to 0.
    - Otherwise, the coins (1-6) are added to the total **only if** they do not exceed 25.
    - If a box contains coins that would cause the total to exceed 25, the coins are discarded, and the player opens another box.
- The game ends when the player reaches exactly 25 coins, and the function returns the number of mystery boxes opened.

Note: no user input required.

Sample output from function call:

```
Gold Coins: 0
Box contains: 4 ... Total now: 4
Box contains: 6 ... Total now: 10
Box contains: 5 ... Total now: 15
Box contains: 6 ... Total now: 21
Box contains: 5 ... Discarded, try again
Box contains: 4 ... Total now: 25
Treasure Chest Full!

< function returns 6 >
```

(15 marks)

Question 10

Extend the game in Q9 where three adventurers now compete to fill two treasure chests each. Each chest must reach exactly 25 coins, and players must avoid bombs that can wipe out their current chest.

You must employ the *openMysteryBox()* function from Q9 to simulate the contents of each mystery box.

Game Rules:

1. Adventurers: The game includes "Lara", "Nico", and "Zane".
2. Initialize a dictionary to store each adventurer's progress across two chests:

   boxOpened = { "Lara": [0, 0], "Nico": [0, 0], "Zane": [0, 0] }

   Each list represents the number of boxes opened for each chest.
   For example: "Lara": [6,8] means Lara opened 6 boxes for the first chest and 8 for the second, totaling 14 boxes.

3. Mystery Boxes: Each box contains either:

   - A random number of coins between 1 and 6, or
   - A bomb (represented by -1), which resets the current chest to 0.

4. Chest Progression:

   - Players fill their first chest to exactly 25 before moving to the second. Record the number of boxes opened for this player into the dictionary.
   - If a coin value would cause the current chest to exceed 25, it is discarded.
   - If a bomb is drawn, the current chest resets to 0.

5. After each player fill up their 2 chests, determine the winner as the player who opened the least number of boxes. Your program must handle the condition that there could be a draw.

Note: no user input required.

Sample run 1:

```
….
…. output from openMysteryBox() for each player ….
….
Game Results:
Lara: Chest 1 = 6, Chest 2 = 8, Total = 14
Nico: Chest 1 = 7, Chest 2 = 9, Total = 16
Zane: Chest 1 = 5, Chest 2 = 10, Total = 15
Winner: Lara with 14 boxes!
```

Sample run 2:

….
…. output from openMysteryBox() for each player ….
….
Game Results:
Lara: Chest 1 = 6, Chest 2 = 8, Total = 14
Nico: Chest 1 = 7, Chest 2 = 7, Total = 14
Zane: Chest 1 = 5, Chest 2 = 10, Total = 15
It's a draw between: Lara, Nico with 14 boxes!

(15 marks)

----- END OF PAPER -----